

docker_class_9(52)

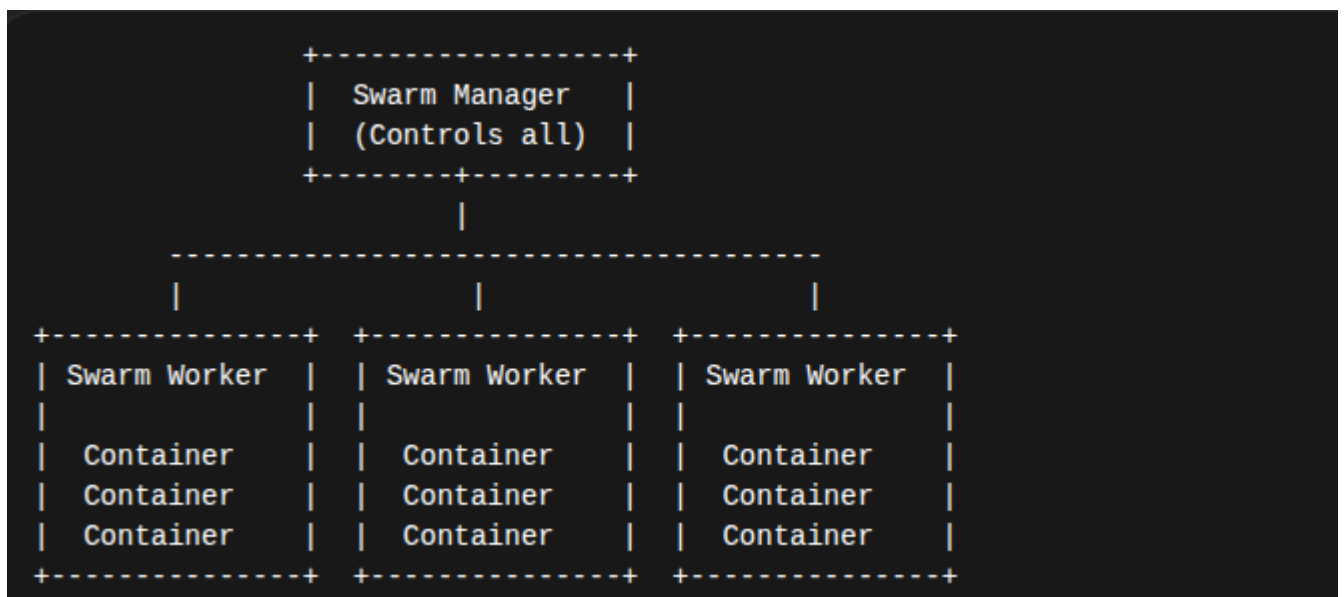
DOCKER SWARM:

In Ansible, all dependencies are installed automatically through playbooks over SSH, but it takes more time. Here, we can complete it much faster.

Orchestration = automated coordination and management of multiple systems, services, or tasks so they work together smoothly. this is like master or leader

Orchestration / cluster both are same

- Docker swarm is an orchestration service within docker that allows us to manage and handle multiple containers at the same time.
- It is a group of servers that runs the docker application.
- It is used to manage the containers on multiple servers.
- This can be implemented by the cluster.
- The activities of the cluster are controlled by a swarm manager, and machines that have joined the cluster is called swarm worker.
 - A group of these servers is called a **cluster**.
 - The cluster is managed by a **Swarm Manager** (controls everything).
 - Other machines in the cluster are called **Worker Nodes** (run containers).
 - It helps with:
 - Scaling containers
 - Load balancing
 - High availability
- Docker Swarm is a tool that manages and runs containers across multiple servers in a cluster automatically. where a manager node controls the system and worker nodes run the applications efficiently.



master-slave in jenkins

manager-worker in docker

A **cluster** is a collection of servers (nodes) and master that work together as a single system.

👉 Workers are **inside** the cluster, not separate from it.

Cluster

```

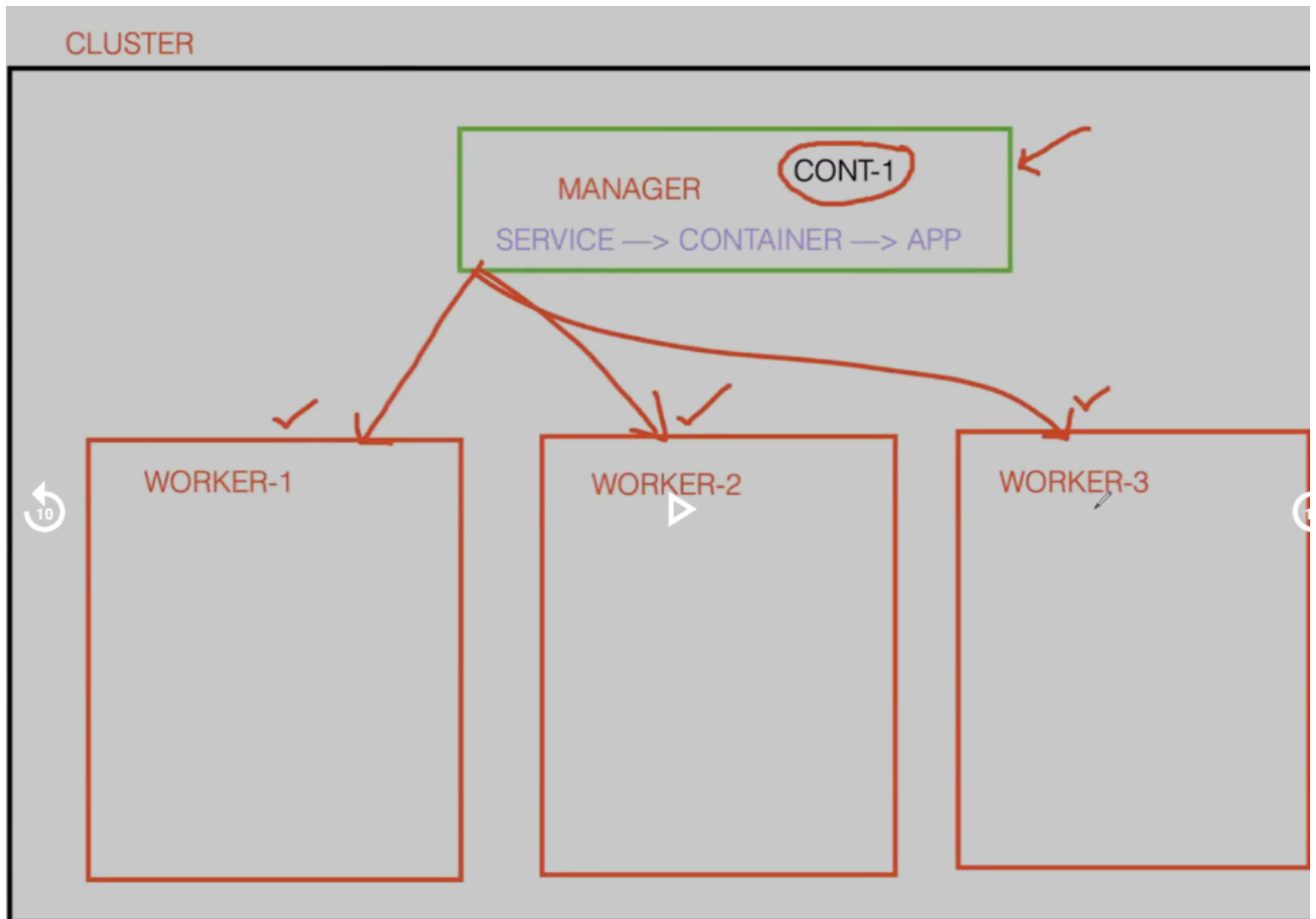
├── Manager
├── Worker
├── Worker
└── Worker
  
```

✅ Cluster = **manager + workers together**

Quick analogy:

- Cluster = company
- Manager = boss
- Workers = employees

swarm will take the application will work every nodes



- Docker Engine helps to create Docker Swarm.
- There are mainly worker nodes and manager nodes.
- The worker nodes are connected to the manager nodes.
- So any scaling or update needs to be done first it will go to the manager node.
- From the manager node, all the things will go to the worker node.
- Manager nodes are used to divide the work among the worker nodes.
- Each worker node will work on an individual service for better performance.

◦ DOCKER SWARM COMPONENTS:

SERVICE: Represents a part of the feature of an application.

TASK: A single part of work.

MANAGER: This manages the work among the different nodes.

WORKER: Which works for a specific purpose of the service.

will setup swam now one is manager two is workers

m1

w1

w1

3 systems install docker

single command worker nodes we can connect to manager

first do in manager

docker swarm init --advertise-addr 172.31.3.72

```
[root@ip-172-31-3-72 ~]# docker swarm init --advertise-addr 172.31.3.72
```

Swarm initialized: current node (wtcoijpzd9kegz5ckdoc7ww2r) is now a manager.

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-
```

```
28o0iv9srmlt755oj2okpg9jzep6yvdpq8rw3n1cyqpmp6lu295-6f66pc9gqgv6zumohtrar5lbnpp  
172.31.3.72:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

```
[root@ip-172-31-3-72 ~]#
```

if we want add workers to swarm we have to run the above command

right now i have run in w1 and w2

```
[root@ip-172-31-6-161 ~]# docker swarm join --token SWMTKN-1-
```

```
28o0iv9srmlt755oj2okpg9jzep6yvdpq8rw3n1tyqpmp6lu295-6f66pc9gqgv6zumohtrar5lbnpp  
172.31.3.72:2377
```

This node joined a swarm as a worker.

in w1 and w2 after run i will see the msg This node joined a swarm as a worker.

nothing will need single command

TO see the list of nodes: docker node ls

Note:include manager will also see and

we can see hostname status.

coming to **Orchestration** easy topics in devops is docker swarm

docker swam never we should create container we create only services

that service will create a container

To create a service : docker service create --name harish --publish 8081:80 nginx

we are giving service name harish next that service it will create container if we want to access so given the ports but we do not need to give container that service will add like a container taken care but additionally it will add id

Note: running in manger node

How to check is service is created: docker service ls

```
[root@ip-172-31-3-72 ~]# docker service create --name mustafa --publish 8081:80 nginx
```

```
vusrdmopbu430j1h62warzzej
```

overall progress: 1 out of 1 tasks

```
1/1: running [=====>]
```

verify: Service converged

```
[root@ip-172-31-3-72 ~]# docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
vusrdmopbu43	mustafa	replicated	1/1	nginx:latest	*:8081->80/tcp

```
[root@ip-172-31-3-72 ~]# docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
--------------	-------	---------	---------	--------	-------

ef673ce649491	nginx:latest	"/docker-entrypoint..."	About a minute ago	Up	About a minute
80/tcp	mustafa.1.rwjmhqq0jnirukh27p029gggyo				

one service have multiple containers

Q1: this container will work only manager also worker Ans: both workers and manager

You created a **Docker Swarm service**, not a normal container:

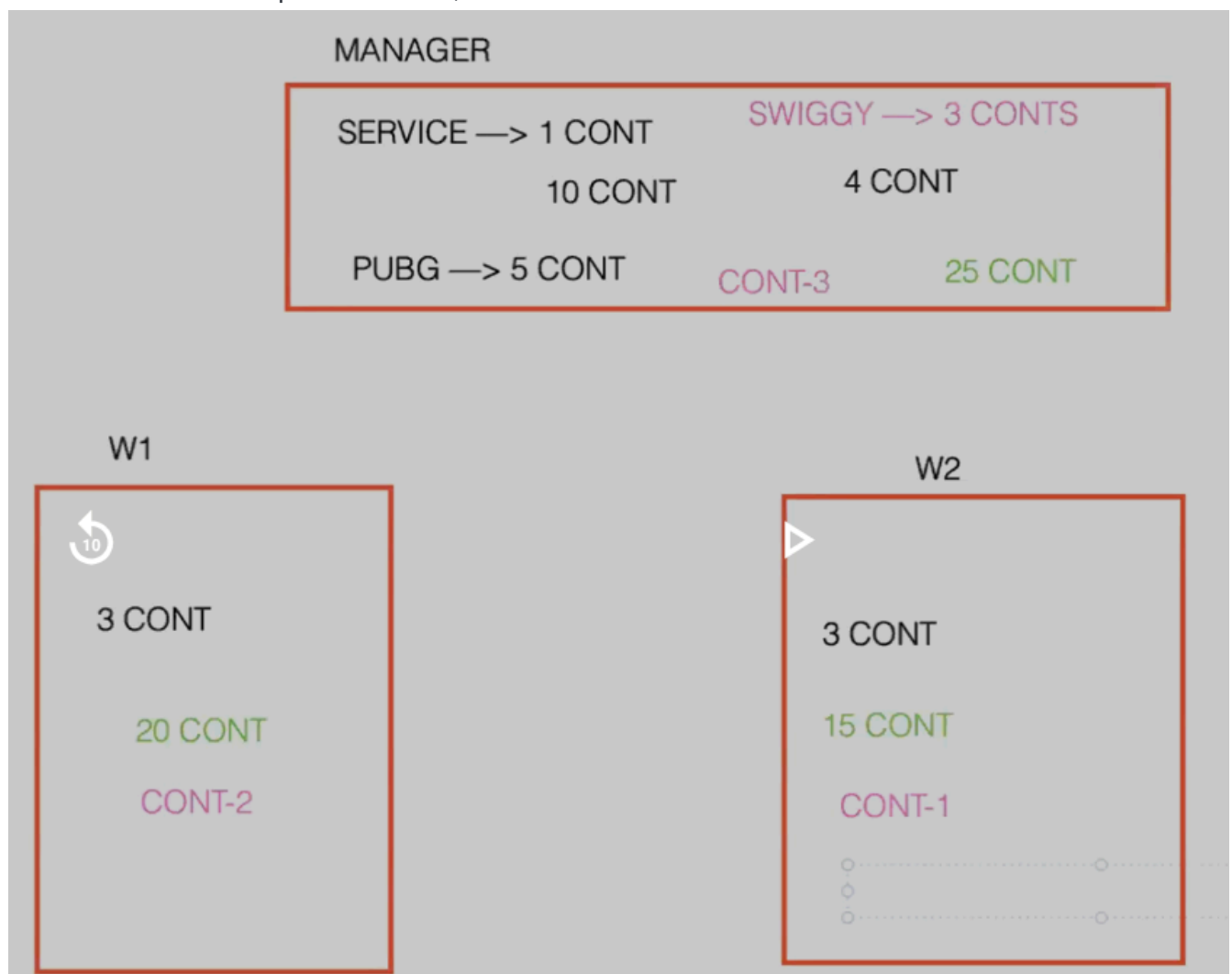
- managed by swarm ✓
- auto-restarts if crashes ✓
- scalable (`--replicas`) ✓

Next Topic: distribution system container based on load

one service if we create 10 containers then it shifts managers 4 containers, worker1 3 containers and worker2 3 containers

managers have 25 containers, worker1 has 10 containers and worker2 has 15 containers already existing

now I am taking swiggy service, this service I want to create 3 containers, 1st priority will go to w2 and second container preference w1, so based on load it will distribute



Note: In manager, if we stop or remove the containers, still the application will work on all 3 servers because of replication. Whenever you remove it, it will create a new container again.

The container may be present in any node in the cluster. If it is not present in one server, it will still work as it may be present on other nodes.

This is a big advantage because if one container is deleted, it will create another container in its place. It may be on any server, but the application will continue to work.

will see now one service how to create multiple containers

```
docker service create --name vamsi --publish 8082:80 --replicas 5 httpd
```

enter

```
docker service ls
```

it will show 5 container has created

```
[root@ip-172-31-3-72 ~]# docker service ls
ID                NAME      MODE      REPLICAS  IMAGE      PORTS
vusrdmopbu43     mustafa   replicated 1/1        nginx:latest *:8081->80/tcp
klj8v8pxo5ld     vamsi     replicated 5/5        httpd:latest *:8082->80/tcp
[root@ip-172-31-3-72 ~]#
```

manager 1 will created w1 will be 2 w2 will be 2

```
[root@ip-172-31-3-72 ~]# docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS          NAMES
04973a31ec21   httpd:latest "httpd-foreground"     About a minute ago Up About a minute 80/tcp         vamsi.3.151dls28eff242ss2mr5esxj7
212c315956e0   nginx:latest "/docker-entrypoint..." 3 minutes ago   Up 3 minutes   80/tcp         mustafa.1.f6d7kngrq0fip05vexxk9xsk
[root@ip-172-31-3-72 ~]#
```

Vamsii.3 means the 3rd container in the manager.

Go to the W2 worker.

If I remove one container, it will be created again.

```
docker service create --name yaswanth --replicas 3 --publish 8083:80 shaikmustafa/cycle
```

This is a public image on Docker Hub.

```
docker ps -a
```

A container with the name "yaswanth" is first created in the manager.

```
[root@ip-172-31-3-72 ~]# docker ps -a
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS         PORTS          NAMES
9786de7fef0d   shaikmustafa/cycle:latest "/docker-entrypoint..." 9 seconds ago   Up 8 seconds   80/tcp         yas$wanth.1.nivtt0np39m3qe14duhagjdfk
04973a31ec21   httpd:latest          "httpd-foreground"     4 minutes ago   Up 3 minutes   80/tcp         vamsi.3.151dls28eff242ss2mr5esxj7
212c315956e0   nginx:latest          "/docker-entrypoint..." 6 minutes ago   Up 6 minutes   80/tcp         mustafa.1.f6d7kngrq0fip05vexxk9xsk
[root@ip-172-31-3-72 ~]#
```

We can check the remaining two servers.

Docker Swarm is powerful.

cat Dockerfile

FROM nginx

COPY ./usr/share/nginx/html/

```
vim index.html
```

```
<h1>This is my web application</h1>
```

```
docker build -t image1 .
```

By seeing this image, can we implement Swarm? Yes.

```
docker service create --name harish --publish 8084:80 --replicas 3 image1
```

```
m = 3
```

```
w1 = 3
```

```
w2 = 3
```

Is it created on all the nodes? Yes.

Answer: No, all the containers will be created only on the manager.

Run this command.

```
[root@ip-172-31-3-72 ~]# docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
457ae923176d	image1:latest	"/docker-entrypoint..."	20 seconds ago	Up 13 seconds	80/tcp	tharun.3.5xplmscqs4ggells9zboqpm3h
751275150a72	image1:latest	"/docker-entrypoint..."	20 seconds ago	Up 13 seconds	80/tcp	tharun.1.bp1fdp701fg2hhrf7liopa7g5
e61753387c17	image1:latest	"/docker-entrypoint..."	40 seconds ago	Up 39 seconds	80/tcp	tharun.2.g5ewphxylijs2j7lqy699hd6
9786de7fef0d	shaikmustafa/cycle:latest	"/docker-entrypoint..."	5 minutes ago	Up 5 minutes	80/tcp	yaswanth.1.nivtt0np39m3gel14duhagjdfk
04973a31ec21	httpd:latest	"httpd-foreground"	9 minutes ago	Up 9 minutes	80/tcp	vamsi.3.151dls28eff242ss2mr5esxj7
212c315956e0	nginx:latest	"/docker-entrypoint..."	11 minutes ago	Up 11 minutes	80/tcp	mustafa.1.f6d7kngrg0fip05vexk9xak

All containers will run on the manager, but the application can be accessed on W1 and W2. When running the command, it shows “no such image” in the logs.

This is because image1 is only present on the manager; it is not available on the worker nodes. Without the image, the container cannot be created.

Earlier, it worked because those were public images available on Docker Hub. Since image1 is not present locally on W1 and W2, the containers and services are rejected on those nodes. Due to this, the service is not created on W1 and W2.

Troubleshoot commands

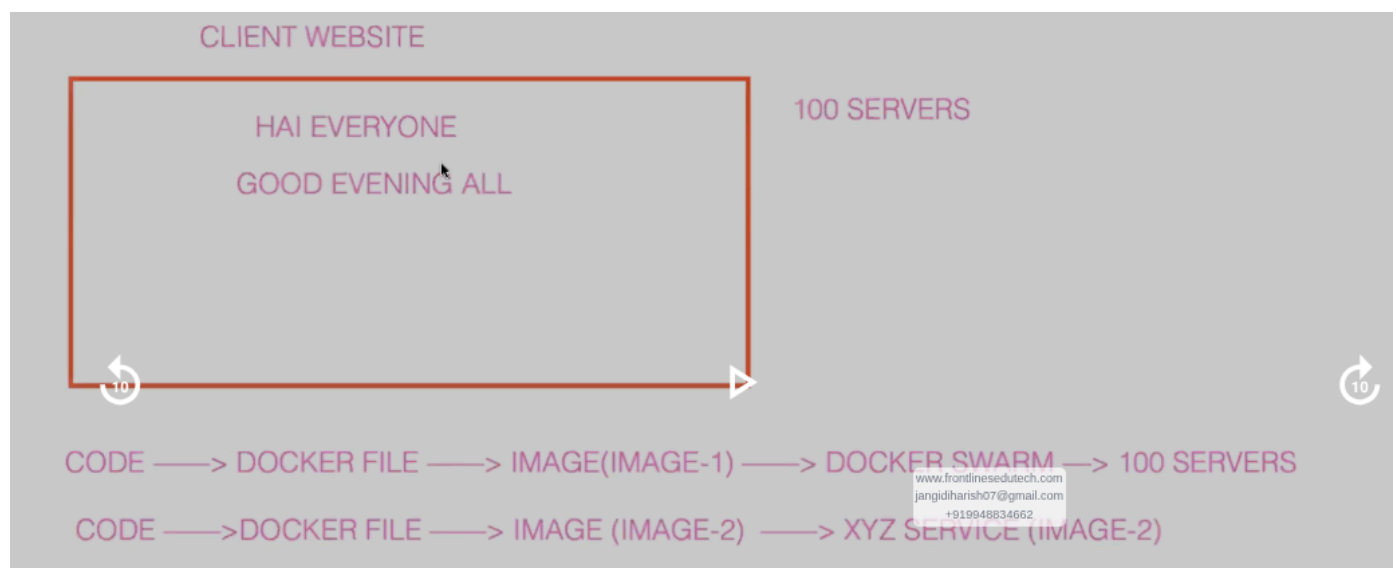
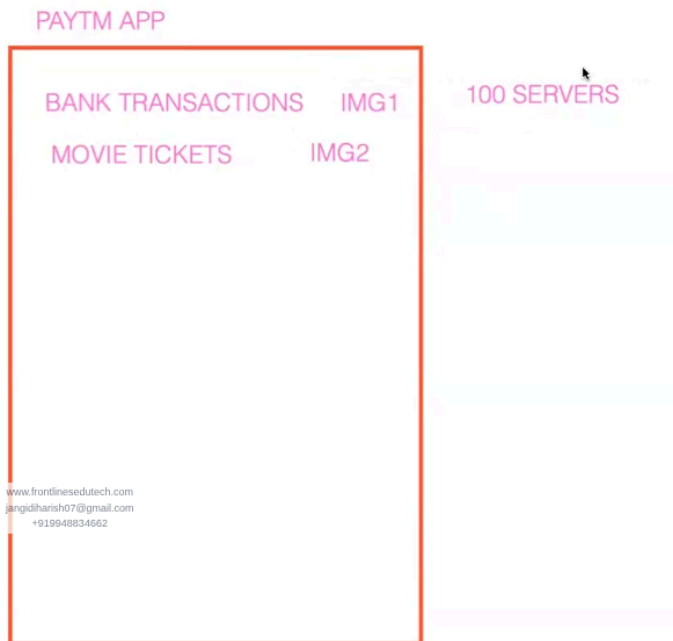
```
[root@ip-172-31-3-72 ~]# docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
vusrdmopbu43	mustafa	replicated	1/1	nginx:latest	*:8081->80/tcp
2k4bab67ckvg	tharun	replicated	3/3	image1:latest	*:8084->80/tcp
klj8v8pxo51d	vamsi	replicated	5/5	httpd:latest	*:8082->80/tcp
yuuhhvzhz6vuo	yaswanth	replicated	3/3	shaikmustafa/cycle:latest	*:8083->80/tcp

If you observe here, the “yaswanth” service (cycle) is running on port 8083. Now the client has changed the business requirement—they no longer want the cycle application; instead, they are asking for a digital marketing website.

This time, I will not deploy a new service again. I will update the existing service.

Example: Initially, we had a bank transaction application using the img1 image, and it was deployed on 100 servers. Later, the client requested an additional feature—movie ticket booking. So now, I will use img2 and deploy it on 100 servers. The img2 image contains both applications, i.e., bank transactions and movie ticket booking.



To see the list of services:

`docker service ls`

Note: If we remove a service, the containers will also be deleted.

`docker service rm service_name`

We will do a practical:

Step 1:

`vi index.html`


```
<h1>Hello everyone</h1>
```

```
cat Dockerfile
```

```
FROM nginx
```

```
COPY . /usr/share/nginx/html/
```

Step 2:

```
docker build -t image1 .
```

Step 3:

Later, we have to push it to Docker Hub.

Step 4:

Creating a service:

```
docker service create --name akhil --publish 8081:80 --replicas 3 image1
```

Right now, we did not push it to Docker Hub, so the image is available only on the manager, not on W1 and W2.

If you run `docker ps -a`, all containers will run only on the manager node.

```
verify: Service converged
[root@ip-172-31-3-72 ~]# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
a77407faff3c   image1:latest  "/docker-entrypoint..." 17 seconds ago Up 10 seconds 80/tcp       akhil.2.ndn8yu9x0yfnlh763rcxjf5ut
5b7fc887eeb5   image1:latest  "/docker-entrypoint..." 17 seconds ago Up 10 seconds 80/tcp       akhil.1.jtwrphwma4e4u8blbcenleavc
55e5c58bc3b9   image1:latest  "/docker-entrypoint..." 38 seconds ago Up 36 seconds 80/tcp       akhil.3.tlw2mx7v6jz9hlzk1rvn09yl9
[root@ip-172-31-3-72 ~]#
```

But on W1 and W2, the page will still work. You can check it using the IP address with the port number. You will see the message: "Hello everyone."

Step 5:

Now the client asks to update "Hello everyone" with "Good evening."

```
vi index.html
```

```
<h1>Hello everyone</h1> <h2>Good evening</h2>
```

Step 6:

Build the image:

```
docker build -t image2 .
```

Step 7:

```
[root@ip-172-31-3-72 ~]# docker service ls
ID            NAME    MODE     REPLICAS  IMAGE          PORTS
gpnzb752p5mv  akhil   replicated 3/3        image1:latest  *:8081->80/tcp
```

we have to update the existing image with new image iamge2

```
docker service update --image:image2 akhil
```

We have to update the existing service with the new image, image2.

```
docker service update --image image2 akhil
```

The new image will be updated.

Now the output will be "Hello everyone" with "Good evening."

If the client asks again, we will edit the index file, build a new image, and update the service again.

Can you write a CI/CD pipeline for this?

We have different versions of images, and we can revert to the previous version using rollback. However, rollback will go only to the immediate previous version.

Let's say we have versions 1, 2, 3, and 4.

We can go from version 4 to 3, but we cannot directly go from 3 to 2. From 3, it will go back to 4.

If we want to go to a specific version, we cannot do it directly in this case. For that, we use Kubernetes.

How to rollback:

```
docker service rollback akhil
```

For local images, sometimes rollback may not work. So, I am creating a service using public images.

1. Create service:

```
docker service create --name mustafa --publish 8082:80 --replicas 3 shaikmustafa/cycle
```

The cycle application will run on all 3 servers. Now the client asks for a digital marketing application.

2. Update image:

```
docker service update --image shaikmustafa/dm mustafa
```

Now it changes to the digital marketing application everywhere.

3. Update again:

```
docker service update --image shaikmustafa/mygame mustafa
```

Now all containers and the manager will be updated with the game application.

In production, the application is accessed in this way:

1st version: cycle

2nd version: digital marketing

3rd version: game

Rollback command:

`docker service rollback mustafa`

Now it will roll back to the digital marketing version.

If I run rollback again, it will go back to the game version.

When we perform a rollback, the old containers are deleted and new ones are started. We can check this using:

`docker ps -a`

Q1:

Assume there are 10 containers in Docker:

- 3 are created manually
- 7 are created by a service

If I want to see only the service-related containers:

`docker service ps mustafa`

To list services:

`docker service ls`

```
[root@ip-172-31-3-72 ~]# docker service ls
ID                NAME        MODE           REPLICAS  IMAGE                                  PORTS
gpnzb752p5mv     akhil       replicated      3/3        image4:latest                        *:8081->80/tcp
w093g9xwxqzi     mustafa     replicated      3/3        shaikmustafa/mygame:latest          *:8082->80/tcp
[root@ip-172-31-3-72 ~]#
```

mustafa service have 3 containers but i want to scale up the services

`docker service scale mustafa=10`

```
[root@ip-172-31-3-72 ~]# docker service scale mustafa=10
mustafa scaled to 10
overall progress: 10 out of 10 tasks
1/10: running [=====>]
2/10: running [=====>]
3/10: running [=====>]
4/10: running [=====>]
5/10: running [=====>]
6/10: running [=====>]
7/10: running [=====>]
8/10: running [=====>]
9/10: running [=====>]
10/10: running [=====>]
verify: Waiting 2 seconds to verify that tasks are stable...
```

Now it will scale up to 10 containers.

So I can check: the manager created 3, worker1 created 4, and worker2 created 4.

We can scale up and scale down also.

Scale up means increasing from 10 containers to 20 containers.

Scale down means reducing from 10 containers to 5 containers.

Scale down command:

```
docker service scale mustafa=5
```

We cannot decide which containers will be deleted; it is handled by the system.

If we want to check the logs of a service:

```
docker service logs mustafa
```

We can see all logs, including who accessed the application.

To inspect the service:

```
docker service inspect mustafa
```

We can get all background details.

To remove the service:

```
docker service rm mustafa
```

Commands summary:

To create a service:

```
docker service create --name service_name --publish 8081:80 --replicas 3 image_name
```

To see the list of services:

```
docker service ls
```

To update the image:

```
docker service update --image image_name service_name
```

To scale:

```
docker service scale service_name=count
```

To get logs:

```
docker service logs service_name
```

To get containers of a particular service:

```
docker service ps service_name
```

To inspect:

```
docker service inspect service_name
```

To remove:

```
docker service rm service_name
```

To see the list of nodes:

```
docker node ls
```

Now, if I want more nodes—currently I have 3 nodes, and I want one more worker (worker3)—I will launch a new instance. Will it automatically join the cluster? No.

First, install Docker and join it to the cluster using the join token. Only then it will be added.

If I run this command again on the manager:

```
docker swarm init --advertise-addr 172.31.3.72
```

It will give a daemon error.

```
[root@ip-172-31-3-72 ~]# docker swarm init --advertise-addr 172.31.3.72
Error response from daemon: This node is already part of a swarm. Use "docker swarm leave" to leave this swarm and join another one.
[root@ip-172-31-3-72 ~]#
```

to generate again token we are added worker3 in manager for that command

```
docker swarm join-token worker
```

will get worker key and we can check `docker node ls` now we have 3 workers

```
[root@ip-172-31-3-72 ~]# docker swarm join-token worker
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-28o0iv9srmlt755oj2okpg9jzpe6yvvdq8rw3nltypqmp6lu295-6f66pc9qaqv6zumohar5lbnp 172.31.3.72:2377

[root@ip-172-31-3-72 ~]# docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
wtcoijpzd9kfg5ckodc7ww2r *	ip-172-31-3-72.us-east-2.compute.internal	Ready	Active	Leader	20.10.25
nsoug25wsd538qyv73lsa3753	ip-172-31-6-161.us-east-2.compute.internal	Ready	Active		20.10.25
of6wvcs12c3kifp43kbm5jnc3	ip-172-31-10-7.us-east-2.compute.internal	Ready	Active		20.10.25
j4um72k5xql10oftn8x17mqu2	ip-172-31-17-137.us-east-2.compute.internal	Ready	Active		20.10.25

```
[root@ip-172-31-3-72 ~]#
```

old one will not expire it will be there

Q2: In a cluster, how do we remove a node?

We are in worker1.

```
docker swarm leave
```

Node left the swarm.

When a node leaves the swarm, the manager will no longer include it in the swarm cluster.

Q3: In the manager, if I run `docker node ls`, it will show 4 nodes. But if you observe, one container is in a down state.

```
[root@ip-172-31-3-72 ~]# docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
wtcoijpzd9kfzg5ckode7ww2r *	ip-172-31-3-72.us-east-2.compute.internal	Ready	Active	Leader	20.10.25
nsoug25wsd538qyv73lsa3753	ip-172-31-6-161.us-east-2.compute.internal	Ready	Active		20.10.25
of6wwcs12c3kifp43kbn5jnc3	ip-172-31-10-7.us-east-2.compute.internal	Down	Active		20.10.25
j4um72k5xql10oftn8x17mqu2	ip-172-31-17-137.us-east-2.compute.internal	Ready	Active		20.10.25

```
[root@ip-172-31-3-72 ~]#
```

After a node leaves, it will go into a down state. Then, after 10 seconds, on the manager we need to run a command to completely remove the node:

```
docker node rm of6wwkjrbksbkjir
```

```
docker node ls
```

Now you will see 3 nodes.

Again, if I want to add worker1 back, we need to use the command:

```
docker swarm join-token worker
```

Copy the token and paste it in worker1.

There is no rule that we can only add workers; we can also add multiple managers.

To get the manager join token:

```
docker swarm join-token manager
```

Now copy this token and paste it in worker2.

In worker2, I am adding it as a manager.

```
[root@ip-172-31-6-161 ~]# docker swarm join --token SWMTKN-1-28o0iv9srmlt755oj2okpq9jzep6yvvdq8rw3nltyqmp6lu295-bw8pdjv1cwp201ztc1mn0ifte 172.31.3.72:2377
This node joined a swarm as a manager.
[root@ip-172-31-6-161 ~]#
```

in one cluster we can have no of nodes and managers also

in manager iam giving docker node ls

```
[root@ip-172-31-3-72 ~]# docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
wtcoijpzd9kfzg5ckode7ww2r *	ip-172-31-3-72.us-east-2.compute.internal	Ready	Active	Leader	20.10.25
x4bv45tfjva91tlp5cjlyj0tj	ip-172-31-6-161.us-east-2.compute.internal	Ready	Active	Reachable	20.10.25
hylkk4lyyy4z1midhl0dastsf	ip-172-31-10-7.us-east-2.compute.internal	Ready	Active		20.10.25
j4um72k5xql10oftn8x17mqu2	ip-172-31-17-137.us-east-2.compute.internal	Ready	Active		20.10.25

```
[root@ip-172-31-3-72 ~]#
```

The leader is the primary manager.

“Reachable” is the secondary manager (backup manager). This is the second position.

What is the difference between a manager and a node?

The manager can view all the nodes and can perform actions like update, delete, scale up, and scale down.

Whereas worker nodes cannot perform these operations. They only run the containers assigned to them by the manager.

NODE COMMANDS:

TO INIT THE SWARM: `docker swarm init --advertise-addr private-ip`

TO SEE LIST OF NODES : `docker node ls`

TO GET MANAGER TOKEN: `docker swarm join-token manager`

TO GET WORKER TOKEN: `docker swarm join-token worker`

TO LEAVE THE NODE FROM SWARM: `docker swarm leave`

TO REMOVE THE NODE: `docker node rm node-id`